Cetus Protocol Web Security Audit Report

Version 1.0

alva-p.xyz

April 8, 2026

Prepared by: alva-p Lead Auditors: - Alvaro Pineda

Table of Contents

- Protocol Summary
- Disclaimer
- Risk Classification
- Audit Details
  - Scope
  - Roles
- Executive Summary
  - Threat Model
  - Issues found
- Findings
  - High
  - Medium
  - Low
  - Informational

Protocol Summary

Cetus is a pioneer DEX (Decentralized Exchange) and concentrated liquidity protocol built on the Sui blockchain network. It provides token swapping, liquidity provision (CLMM and DLMM pools), margin trading with up to 3x leverage, DCA (Dollar Cost Averaging), limit orders, vaults, and cross-chain bridging via LI.FI integration.

The web application at `app.cetus.zone` serves as the primary user interface, built with Vite + React and deployed on Netlify with CloudFront CDN. The backend API at `api-sui.cetus.zone` provides pool data, swap routing, margin trading, and user portfolio information.

Key components audited:

- **Frontend**: `app.cetus.zone` (Vite + React SPA on Netlify + CloudFront)
- **Legacy Frontend**: `v1.cetus.zone` (Nuxt.js/Vue on Netlify + CloudFront)
- **API Backend**: `api-sui.cetus.zone` (REST API, no authentication)
- **API Backend (CF-protected)**: `api-sui-cf.cetus.zone` (Cloudflare challenge)
- **Bridge UI**: `bridge.cetus.zone` (Netlify)
- **Bridge Validator**: `mainnet-bridge.validator.cetus.zone` (Caddy)
- **Archive/Assets**: `archive.cetus.zone` (Netlify, partial 404)
- **Additional subdomains**: `terminal`, `optimize`, `deepbook`, `vesting` (all Netlify)

Disclaimer

The alva-p team makes all effort to find as many vulnerabilities in the code in the given time period, but holds no responsibilities for the findings provided in this document. A security audit by the team is not an

endorsement of the underlying business or product. The audit was time-boxed and the review of the web application was solely on the security aspects of the frontend, API, and infrastructure.

Risk Classification

Table 1: Likelihood

		Impact		
		High	Medium	Low
	High	H	H/M	M
Likelihood	Medium	H/M	M	M/L
	Low	M	M/L	L

Audit Details

- **Date:** April 8, 2026
- **Target:** `*.cetus.zone` (Web application and API)
- **Audit Method:** Manual reconnaissance, JavaScript bundle analysis, API endpoint testing, DNS/infrastructure analysis
- **Tools Used:** curl, dig, crt.sh, manual JS analysis, browser DevTools

Scope

Target	Type	Scope Level
`app.cetus.zone`	Web Application	Critical
`*.cetus.zone`	Subdomains	Medium
`api-sui.cetus.zone`	REST API	Medium

Assets analyzed:

- HTML source of `app.cetus.zone`
- JavaScript bundles: `index-BB9-hnFI.js` (7.1MB), `vendor-CL3dImV2.js` (646KB), `cross-BeDO33W5.js` (1.6MB)
- DNS records for 19 subdomains via Certificate Transparency (crt.sh)
- API endpoints (90+ routes discovered)
- HTTP security headers across all subdomains

Roles

- **User** – Any visitor connecting a Sui wallet to perform swaps, provide liquidity, or trade on margin.
- **API Consumer** – Any client making requests to `api-sui.cetus.zone` (no authentication required).
- **Attacker** – External actor attempting to exploit web vulnerabilities, takeover subdomains, or abuse API endpoints.

Executive Summary

The Cetus web application presents several security vulnerabilities across its infrastructure, frontend, and API layers. The most significant finding is an **API URL injection** parameter in the JavaScript bundle that allows full API hijacking on non-primary subdomains, enabling transaction manipulation through crafted links.

Additionally, the main application loads external JavaScript from a partially abandoned subdomain **without Subresource Integrity (SRI)**, creating a supply chain attack vector. The API layer exposes **user wallet balances and margin trading activity** without authentication, enabling financial surveillance and front-running. Sensitive credentials including API keys, development environment URLs, and private RPC endpoints are hardcoded in the production JavaScript bundle. Two subdomains have orphaned CloudFront CNAME records, though CloudFront's domain ownership validation limits their exploitability.

Threat Model

- **Protocol Type:** DEX (Decentralized Exchange) with concentrated liquidity, margin trading, and cross-chain bridging
- **Primary Attack Surface:** Web frontend (SPA), REST API, DNS/infrastructure
- **Trust Boundaries:** Frontend-to-API (no auth), Frontend-to-external-subdomains (JS loading), DNS-to-CDN (CNAME records)
- **Key Risk:** API URL injection enabling API manipulation in a financial application where users sign blockchain transactions

Issues found

Table 3: Issues found summary

Severity	Number of issues found
High	0
Medium	4
Low	3
Info	4
Total	11

Findings

High

No high severity issues found.

Medium

[M-1] `apiUrl` query parameter injection enables full API hijacking on non-primary subdomains

Severity: Medium (Impact: High, Likelihood: Low)

Location: JavaScript bundle `index-BB9-hnFI.js`

Description:

The application reads a `apiUrl` query parameter from the URL and uses it as the base URL for **all API calls**:

```
function getQueryParam(Cu) {
  const Au = new URLSearchParams(window.location.search);
  return window.location.host === "app.cetus.zone" ? "" : Au.get(Cu);
}
let apiUrl = getQueryParam("apiUrl") || "";

// Usage:
// aggregatorUrl = `${apiUrl || "https://api-sui.cetus.zone"}/router_v3`
// cetus_api: apiUrl || "https://api-sui.cetus.zone"
```

The check `window.location.host === "app.cetus.zone"` only disables this on the main domain. On any other subdomain (e.g., `bridge.cetus.zone`, `terminal.cetus.zone`, or any phishing clone), the parameter is fully functional, allowing an attacker to redirect all API calls to a malicious server.

Attack Path:

1. Attacker hosts a copy of the Cetus app on any non-primary subdomain or phishing domain
2. Attacker crafts URL: `https://<subdomain>.cetus.zone/swap?apiUrl=https://attacker-api.com`
3. Victim opens the link – the app loads normally
4. ALL API calls (swap routing, prices, pool data) go to `attacker-api.com`
5. Attacker's API returns manipulated swap routes (e.g., routing through attacker-controlled pools)
6. Victim signs a transaction believing it is a normal swap
7. Funds are drained to the attacker

Impact:

- Complete API hijacking on non-primary subdomains enabling transaction manipulation
- Swap route manipulation leading to fund theft
- Price manipulation affecting user trading decisions

Proof of Concept:

```
# Verify apiUrl injection in source
$ curl -s https://app.cetus.zone/assets/index-BB9-hnFI.js \
  | grep -oP '.{0,50}apiUrl.{0,80}' | head -3
# Returns: getQueryParam function + usage as API base URL

# The check only protects app.cetus.zone:
# window.location.host === "app.cetus.zone" ? "" : Au.get(Cu)
# Any other host allows the parameter
```

Recommended Mitigation:

1. **Immediate:** Remove the `apiUrl` query parameter functionality entirely

2. **Short-term:** If needed for development, restrict it to `localhost` only via build-time flags
3. **Long-term:** Hardcode the API base URL and remove all dynamic override mechanisms

[M-2] External JavaScript loaded without Subresource Integrity from subdomain with permissive CORS

Severity: Medium (Impact: High, Likelihood: Low)

Location: `app.cetus.zone` HTML source, `archive.cetus.zone` server

Description:

The main application loads a charting library from a different subdomain without integrity verification:

```
<script src="https://archive.cetus.zone/assets/charting_library_new/
  charting_library.standalone.js"></script>
```

No `integrity` or `crossorigin` attributes are present. The serving subdomain `archive.cetus.zone`:

- Points to Netlify (`splendorous-snickerdoodle-23161b.netlify.app`)
- Homepage returns 404 (partial abandonment)
- JavaScript assets still serve (200 OK, 54,925 bytes)
- CORS headers are fully permissive: `Access-Control-Allow-Origin: *`
- `transfer.cetus.zone` chains via CNAME to `archive.cetus.zone`

If the Netlify site is delinked from the domain or the Netlify account is compromised, an attacker could serve malicious JavaScript that executes in the context of `app.cetus.zone`.

Impact:

- Full XSS on `app.cetus.zone` affecting all DEX users
- Access to wallet connections (WalletConnect sessions)
- Transaction manipulation before user signing
- Session data theft

Proof of Concept:

```
# Script tag without SRI
$ curl -s https://app.cetus.zone | grep "archive.cetus.zone"
<script src="https://archive.cetus.zone/assets/charting_library_new/
  charting_library.standalone.js"></script>

# Permissive CORS
$ curl -sI https://archive.cetus.zone/assets/charting_library_new/\
  charting_library.standalone.js | grep access-control
access-control-allow-origin: *

# Homepage 404 but JS active
$ curl -s -o /dev/null -w "%{http_code}" https://archive.cetus.zone/
404
$ curl -s -o /dev/null -w "%{http_code}" https://archive.cetus.zone/\
```

```
  assets/charting_library_new/charting_library.standalone.js
200
```

Recommended Mitigation:

1. Add SRI hash to the script tag with `crossorigin="anonymous"`
2. Move the charting library to `app.cetus.zone/assets/` (same origin)
3. Restrict CORS on `archive.cetus.zone` to only `app.cetus.zone`

[M-3] Unauthenticated API exposes wallet balances and margin trading positions enabling front-running

Severity: Medium (Impact: Medium, Likelihood: High)

Location: `api-sui.cetus.zone` endpoints `/v3/sui/profile/coin`, `/v3/sui/margin_trading/tx`, `/v3/sui/margin_trading/market`

Description:

Multiple API endpoints expose sensitive user financial data without authentication:

Wallet Balance Enumeration (IDOR): `POST /v3/sui/profile/coin` returns complete wallet balances for any address:

```
POST https://api-sui.cetus.zone/v3/sui/profile/coin
{"address":"0xd26567..."}
# Response: All tokens, balances, TVL in USD
```

Real-Time Margin Trading Leak: `GET /v3/sui/margin_trading/tx` returns all recent margin trades from all users without any filter:

```
{
  "position_id": "0x17319...",
  "type": "open_position",
  "tokens": [{"symbol":"SUI","amount":"8000.16","amount_usd":"$7582"}],
  "new_leverage": 3.004
}
```

Market Position Data: `GET /v3/sui/margin_trading/market` exposes exact position counts (123 longs, 49 shorts in SUI/USDC), total values, and max leverage.

All endpoints respond with `Access-Control-Allow-Origin: *`, allowing any website to read this data cross-origin.

Impact:

- Financial surveillance of any wallet on the platform
- Real-time front-running of margin trading positions
- Whale profiling for market manipulation
- Any website can build an unauthorized Cetus trading tracker

Proof of Concept:

```
# Read any wallet's balances
$ curl -s -X POST "https://api-sui.cetus.zone/v3/sui/profile/coin" \
  -H "Content-Type: application/json" \
  -d '{"address":"0xd265672730b0540ffd3569530682a0f02ef984b7..."}'
# Returns: TVL $473.93, SUI: 500B tokens, NS: 57267, BLUE: 3

# Read ALL margin trading activity
$ curl -s "https://api-sui.cetus.zone/v3/sui/margin_trading/tx" \
  | python3 -m json.tool | head -20
# Returns: All recent trades with amounts, leverage, timestamps

# Verify CORS allows cross-origin reads
$ curl -sH "Origin: https://evil.com" -I \
  "https://api-sui.cetus.zone/v3/sui/margin_trading/tx" \
  | grep access-control
Access-Control-Allow-Origin: *
```

Recommended Mitigation:

1. Add authentication to profile and margin trading endpoints
2. Implement rate limiting on all API endpoints
3. Restrict CORS to `app.cetus.zone` and authorized origins
4. Require `address` parameter on `/margin_trading/tx` and scope to authenticated user

[M-4] Dangling CNAME on Netlify with permissive CORS headers

Severity: Medium (Impact: Medium, Likelihood: Medium)

Location: DNS records for `archive.cetus.zone` and `transfer.cetus.zone`

Description:

`archive.cetus.zone` has a CNAME record pointing to `splendorous-snickerdoodle-23161b.netlify.app`. The site is partially abandoned (homepage returns 404) but still serves assets. It responds with fully permissive CORS headers:

```
access-control-allow-origin: *
access-control-allow-methods: GET, POST, OPTIONS
access-control-allow-headers: Content-Type, Authorization
```

`transfer.cetus.zone` chains via CNAME to `archive.cetus.zone`, inheriting the same risk.

Impact:

- If the Netlify site binding is lost, subdomain takeover becomes possible
- Permissive CORS on a `cetus.zone` subdomain could be leveraged for cross-origin attacks
- Combined with FINDING M-1, this amplifies the supply chain risk

Recommended Mitigation:

1. Restrict CORS headers to specific allowed origins

2. Either fully maintain or fully decommission `archive.cetus.zone`
3. Remove the `transfer.cetus.zone` CNAME chain

Low

[L-1] API key and private RPC endpoints exposed in production JavaScript bundle

Severity: Low (Impact: Medium, Likelihood: Low)

Location: `app.cetus.zone/assets/index-BB9-hnFI.js`

Description:

The production JavaScript bundle contains hardcoded sensitive information:

LI.FI API Key:

```
VITE_LIFI_API_KEY="43d28e4f-5fbb-42cc-b8a8-9fe73f40766a.
  41849565-bd75-4f81-b970-6e6184fc8cf5"
```

Private RPC Endpoints (paid services):

```
https://cetus-suimain-e31f.mainnet.sui.rpcpool.com
https://cetus-solanam-e396.mainnet.rpcpool.com
https://cetus-pythnet-a648.mainnet.pythnet.rpcpool.com/hermes
https://cetus-mainnet-endpoint.blockvision.org
https://sui-mainnet-endpoint.blockvision.org
```

Impact:

- Unauthorized API usage under Cetus's LI.FI account
- Cost generation on Cetus's paid RPC accounts
- Rate limit exhaustion

Recommended Mitigation:

1. Rotate the LI.FI API key and move to server-side proxy
2. Use public RPC endpoints in client code; proxy private RPCs through backend

[L-2] Development environment publicly accessible

Severity: Low (Impact: Medium, Likelihood: Low)

Location: `api-sui.devcetus.com`

Description:

The JavaScript bundle exposes development API URLs. The dev environment at `api-sui.devcetus.com` is publicly accessible and responds with data (42 testnet pools). Development environments may have weaker security controls.

Recommended Mitigation:

1. Restrict dev environment access via VPN, IP whitelist, or authentication
2. Remove dev URLs from production bundles via build-time environment variables

[L-3] Bridge validator exposes version and public key

Severity: Low (Impact: Low, Likelihood: Low)

Location: `mainnet-bridge.validator.cetus.zone/ping`

Description:

The `/ping` endpoint returns:

```
{
  "version": "1.68.1-3c0f387ebb40",
  "metrics_pubkey": "ecjcVH/EkQHNM2XokiDPg4bk85QuurRQGy2UScv4Spg="
}
```

This exposes the exact software version (enabling CVE lookup) and a partial commit hash.

Recommended Mitigation:

Remove version and key information from public endpoints, or restrict `/ping` to internal networks.

Informational

[I-1] Orphaned CloudFront CNAME records on pump-api and pump-static subdomains

Two subdomains have DNS CNAME records pointing to Amazon CloudFront distributions that no longer exist:

Subdomain	CNAME Target	Status
`pump-api.cetus.zone`	`d1s3wj7ljr237l.cloudfront.net`	Does not resolve (distribution deleted)
`pump-static.cetus.zone`	`d2gibvksv3ebbt.cloudfront.net`	Does not resolve (distribution deleted)

While these represent dangling CNAME records, AWS CloudFront has implemented domain ownership validation since 2022, requiring proof of DNS control or a valid SSL certificate to claim alternate domain names. This significantly limits the practical exploitability of these orphaned records.

Recommended Mitigation: Remove the dangling DNS CNAME records to maintain DNS hygiene and eliminate any residual risk.

[I-2] Conflicting CSP and X-Frame-Options headers on app.cetus.zone

The application sends contradictory framing headers:

- `content-security-policy: frame-ancestors 'self' https://sui.m-safe.io;`
- `x-frame-options: DENY`

In modern browsers, CSP takes precedence, allowing `sui.m-safe.io` to embed the application. This is likely intentional for multi-sig wallet integration but the X-Frame-Options header is misleading.

[I-3] `dangerouslySetInnerHTML` used in tooltip components

The React application uses `dangerouslySetInnerHTML` in PopoverBody components for rendering tooltip strings. If any user-controllable data reaches these tooltips as a string, it could result in stored XSS. No direct injection path was confirmed.

[I-4] Google Analytics tracking ID exposed

The tracking ID `G-GF4BQNDDKR` is hardcoded in the HTML. While this is standard practice, it allows third parties to identify the analytics property.

Reconnaissance Summary

Subdomain Enumeration

19 subdomains discovered via Certificate Transparency (crt.sh):

Subdomain	Status	Server	Notes
app.cetus.zone	200	Netlify + CloudFront	Primary target
www.cetus.zone	200	Netlify	Landing page
v1.cetus.zone	200	Netlify + CloudFront	Legacy (Nuxt.js)
bridge.cetus.zone	200	Netlify	Bridge UI
deepbook.cetus.zone	200	Netlify	DeepBook UI
terminal.cetus.zone	200	Netlify	Trading terminal
optimize.cetus.zone	200	Netlify	Optimizer
vesting.cetus.zone	200	Netlify	Token vesting
archive.cetus.zone	404	Netlify	Partially abandoned
mainnet-bridge.validator	200	Caddy	Validator node
testnet.deepbookv3	200	Netlify	Testnet
api-sui-cf.cetus.zone	403	Cloudflare	API (CF protected)
pump-api.cetus.zone	Dead	CloudFront (deleted)	Orphaned CNAME (limited risk)
pump-static.cetus.zone	Dead	CloudFront (deleted)	Orphaned CNAME (limited risk)
transfer.cetus.zone	Chain	CNAME to archive	Inherited risk

API Endpoints Discovered

90+ API routes extracted from JavaScript bundles, including:

- `/v3/sui/clmm/*` – CLMM pool operations
- `/v3/sui/margin_trading/*` – Margin trading
- `/v3/sui/profile/*` – User profiles
- `/v3/sui/deepbookv3/*` – DeepBook integration

- `/v3/sui/dlmm/*` – DLMM pools
- `/v3/sui/dca/*` – DCA operations
- `/v3/sui/vaults/*` – Vault operations
- `/router_v3` – Swap routing